

I'm going to run on my tablet which is going to be just the basic PC desktop format and then I'm also going to install one on my phone and that way I can utilise it there when I'm using my Claude code cli Claude code cli but then I still be able to open up my Linux environment desktop on my my tablet is going to show and then I can open a remote control session that's running on my phone as well on the browser through the Debian user interface

That is a fantastic, highly efficient ecosystem layout. You are essentially turning your phone into a **headless background server** that handles heavy automation and your **Claude Code CLI**, while your tablet acts as your primary, large-screen **frontend display** where you can orchestrate everything. [1]

To pull this off without VNC, you will use **Termux:X11** natively on your tablet, and a tool called **noVNC (which serves a Linux desktop over a web browser)** on your phone. [2, 3]

Here is exactly how to set up this cross-device workstation:

Phase 1: Setting up the Tablet (Your Main Native Screen)

Because your tablet is your main interface, you want maximum speed. We will use **Termux:X11** here for lag-free, hardware-accelerated desktop rendering. [3]

1. **Install the prerequisites:** Grab [Termux \(F-Droid\)](#) and the latest companion APK from the [Termux:X11 GitHub Releases](#).
2. **Install DroidDesk / Desktop Packages:** Open Termux on your tablet and configure your basic PC desktop environment:

```
pkg update && pkg upgrade -y
pkg install x11-repo -y
# Install DroidDesk to automatically generate your clean layout and app store
pkg install git python -yone https://github.com && cd DroidDesk
chmod +x setup.sh && ./setup.sh
```

3. **Launch:** Turn on the Termux:X11 Android app to sit in the background, run droiddesk (or your layout startup command) in Termux, and enjoy your native, smooth tablet PC desktop. [4]

Phase 2: Setting up the Phone (Your Headless Server & Claude CLI)

Your phone will run the heavy backend lifting. It will run **Debian via PRoot**, run your node-based **Claude Code CLI**, and broadcast its desktop over your local Wi-Fi network using an HTTP browser server. [3, 5]

1. Install Debian & Node.js (for Claude Code)

Open Termux on your phone and deploy a Debian distribution:

```
pkg update && pkg upgrade -y
pkg install proot-distro -y
proot-distro install debian
proot-distro login debian
```

Now that you are inside your Debian environment, install Node.js and the official **Anthropic Claude Code CLI**: [5, 6]

```
apt update && apt upgrade -y
apt install nodejs npm git -y
npm install -g @anthropic-ai/claude-code
```

(You can now call your AI agent inside your codebase by typing `claude`). [7]

2. Install the Browser Desktop Server (noVNC)

While still inside your phone's Debian shell, install a lightweight environment (like XFCE) alongside novnc to stream the UI as an interactive website: [3]

```
apt install xfce4 xfce4-goodies tightvncserver novnc -y
```

3. Boot the Phone's Server

Run this command on your phone to launch the environment and start broadcasting it to your local network:

```
vncserver :1
websockify --web /usr/share/novnc 8080 localhost:5901
```

Finally, check your phone's local IP address by opening a separate Termux tab and typing `ifconfig` (it will look something like 192.168.1.5).

Phase 3: The Unified Workflow

Now that both systems are humming, you can link them up seamlessly:

